Complete Full-Stack Development Playbook

70+ Sequential Prompts for Building Production-Ready Web Applications

How to Use This Playbook

For Antigravity IDE: Use these prompts sequentially, one by one For Other Tools: Adapt prompts to your development environment Customization: Replace [Project Name] with your actual project name

1. Project Bootstrap & Structure

Initialize a full-stack project for [Project Name]:

Backend:

- Node.js + TypeScript + Express
- Folder structure: src/ (routes, controllers, services, models, utils, config)
- Package.json with dependencies: express, typescript, cors, helmet, dotenv
- tsconfig.json optimized for Node.js
- .env.example with key variables
- README.md with setup instructions

Frontend:

- React + TypeScript + Vite (or Next.js with App Router)
- Folder structure: src/ (components, pages, hooks, services, utils, types, styles)
- Tailwind CSS configured
- Package.json with dependencies
- .env.example

Include:

- .gitignore for both parts
- ESLint + Prettier configs
- Development scripts
- Basic health check endpoint: GET /api/health

2. Environment Configuration

Set up environment configuration for [Project Name]:

Backend .env:

- NODE_ENV (development/production)
- PORT (default 5000)
- DATABASE_URL (MongoDB Atlas connection string)
- JWT_SECRET (secure random string)
- JWT_REFRESH_SECRET
- CORS_ORIGIN (frontend URL)

Frontend .env:

- VITE_API_URL (backend URL)
- VITE_APP_NAME ([Project Name])
- VITE_ENVIRONMENT

Create:

- .env.example templates
- Config loader utility
- Environment validation on startup
- Separate configs for dev/staging/prod

3. Database Connection & Setup

Set up database connection for [Project Name]:

MongoDB:

- Connect to MongoDB Atlas using Mongoose
- Create reusable database connection module
- Connection error handling and retry logic

- Mongoose global configuration
- Database connection logs

OR PostgreSQL:
- Connect using Prisma ORM
- Generate Prisma schema
- Database migration setup
- Connection pooling

Include:
- Connection status endpoint
- Graceful shutdown handling
- Connection timeout configuration

4. Base API Architecture

Implement RESTful API architecture for [Project Name]:

Structure:
- Express router with API versioning (/api/v1/)
- Controller-Service-Model pattern
- Business logic separated from routing
- Request/Response standardization

Create sample endpoints:
- GET /api/v1/health (health check with DB status)
- Standard response format:

```
{
  success: boolean,
  data: any,
  message: string,
```

```
  error?: object
}
```

Include:

- Route organization by feature
- Async error wrapper utility
- HTTP status code constants

5. User Schema Design

Design User schema for [Project Name]:

MongoDB (Mongoose):

- email (unique, required, lowercase, indexed)
- password (hashed, required)
- name (required)
- role (enum: user/admin, default: user)
- isActive (boolean, default: true)
- emailVerified (boolean, default: false)
- lastLogin (Date)
- createdAt, updatedAt (timestamps)

OR PostgreSQL (Prisma):

- Same fields with appropriate types
- Indexes on email
- Cascading rules

Include:

- Email validation
- Password strength requirements
- User profile extension (separate table/schema)

6. Authentication System - Backend

Implement JWT authentication APIs for [Project Name]:

Endpoints:

- POST /api/v1/auth/signup
  - Validate email, password, name
  - Check email uniqueness
  - Hash password with bcrypt (12 rounds)
  - Create user record
  - Generate JWT tokens (access + refresh)
  - Return user data + tokens

- POST /api/v1/auth/login
  - Validate credentials
  - Compare hashed password
  - Check isActive status
  - Update lastLogin
  - Generate tokens
  - Return user data + tokens

- POST /api/v1/auth/refresh
  - Validate refresh token
  - Generate new access token

- POST /api/v1/auth/logout
  - Invalidate refresh token

Security:

- JWT access token: 15 min expiry

- JWT refresh token: 7 days expiry
- Store refresh tokens in database
- Rate limiting on auth endpoints

7. Password Security & Hashing

Implement secure password handling for [Project Name]:

Using bcrypt:
- Hash passwords before saving (salt rounds: 12)
- Compare passwords securely during login
- Never store plain text passwords
- Never return passwords in API responses

Password Requirements:
- Minimum 8 characters
- At least 1 uppercase letter
- At least 1 number
- At least 1 special character

Include:
- Password strength validator
- Pre-save hooks for hashing
- Secure password comparison method
- Password update functionality

8. Auth Middleware & Route Protection

Create JWT authentication middleware for [Project Name]:

Middleware:
- Extract token from Authorization header (Bearer token)

- Verify JWT token
- Decode user ID from token
- Attach user data to request object (req.user)
- Handle token expiration
- Handle invalid tokens

Protected Route Wrapper:
- requireAuth middleware for private routes
- requireRole(['admin']) for role-based access
- Error responses for unauthorized access

Apply to routes:
- Public: /auth/*
- Protected: /api/v1/users/*, /api/v1/[entity]/*

9. Global Error Handling
Implement centralized error handling for [Project Name]:

Backend:
- Custom AppError class with statusCode
- Async error wrapper (catchAsync)
- Global error middleware
- Environment-specific error responses:
  - Development: full error stack
  - Production: user-friendly messages
- Error logging to console/file
- Validation error formatting
- JWT error handling

Response Format:

```
{
  success: false,
  message: "Error message",
  error: {
    statusCode: 400,
    details: [...]
  }
}
```

Common Error Types:
- ValidationError (400)
- AuthError (401)
- ForbiddenError (403)
- NotFoundError (404)
- ServerError (500)

10. API Testing & Documentation

Set up API testing for [Project Name]:

Create Postman/Thunder Client Collection:

Folders:

1. Health
   - GET /api/health

2. Authentication
   - POST /api/v1/auth/signup (with sample payload)
   - POST /api/v1/auth/login
   - POST /api/v1/auth/refresh
   - POST /api/v1/auth/logout

3. Users

- GET /api/v1/users/me (requires auth)
- PUT /api/v1/users/me

Include:

- Environment variables (baseURL, token)
- Pre-request scripts for token injection
- Test assertions
- Sample request bodies
- Expected responses

Optional: Swagger/OpenAPI documentation setup

11. User Registration Flow

Complete user registration system for [Project Name]:

Backend Enhancement:

- Add email verification fields to User schema
- Generate verification token on signup
- Send verification email (template ready)
- Don't auto-verify on signup

Frontend - Signup Page:

- Responsive registration form
- Fields: name, email, password, confirmPassword
- Real-time validation
- Password strength indicator
- Terms & conditions checkbox
- Form submission with loading state

- Success message and redirect

Validation:

- Email format check
- Password match check
- All required fields
- Async email availability check

12. Login System & Frontend

Implement complete login functionality for [Project Name]:

Frontend - Login Page:

- Email/username + password fields
- Show/hide password toggle
- Remember me checkbox (optional)
- Client-side validation
- Submit with loading state
- Error handling (wrong credentials, unverified email)
- Success redirect to dashboard
- "Forgot Password?" link

Backend Integration:

- Connect to POST /api/v1/auth/login
- Store JWT tokens securely (httpOnly cookies or localStorage)
- Set authentication header for future requests

Security:

- Rate limit login attempts (frontend feedback)
- Account lockout after failed attempts
- Clear error messages without revealing user existence

13. Password Reset Flow

Create password reset functionality for [Project Name]:

Backend:

- POST /api/v1/auth/forgot-password
  - Generate secure reset token
  - Save token hash + expiry (1 hour) to database
  - Send reset email with link

- POST /api/v1/auth/reset-password
  - Validate token and expiry
  - Validate new password strength
  - Hash and update password
  - Invalidate reset token
  - Send confirmation email

Frontend:

- Forgot Password page (email input)
- Reset Password page with token from URL
- Token validation on page load
- New password + confirm password fields
- Password strength indicator
- Success message and redirect to login

Email Templates:

- Password reset request email
- Password reset confirmation email

14. User Profile Management

Build user profile functionality for [Project Name]:

Backend:

- GET /api/v1/users/me (get current user profile)
- PUT /api/v1/users/me (update profile)
- PATCH /api/v1/users/me/avatar (update avatar)

Updateable Fields:

- name
- bio (max 500 chars)
- phone
- location (city, country)
- dateOfBirth
- socialLinks (optional)

Frontend:

- Profile view page
- Profile edit form (pre-filled)
- Avatar upload with preview
- Form validation
- Save/Cancel buttons
- Success notifications
- Optimistic UI updates

Validation:

- Required fields
- Phone format validation
- Date of birth validation (age 13+)
- URL format for social links

15. Role-Based Access Control (RBAC)

Implement RBAC system for [Project Name]:

Backend:
- Add 'role' field to User schema (enum: user, admin)
- Create role checking middleware:
  - requireRole(['admin'])
  - requireRole(['admin', 'moderator'])
- Apply to admin routes

Admin Routes:
- GET /api/v1/admin/users (list all users)
- PUT /api/v1/admin/users/:id/role (change user role)
- DELETE /api/v1/admin/users/:id (delete user)

Frontend:
- Role-based route protection
- Conditional UI rendering based on role
- useAuth hook with role checking
- Admin panel access (admin-only)

Features:
- Default role on signup: 'user'
- Only admins can change roles
- Prevent last admin from being demoted

16. User Dashboard

Create user dashboard for [Project Name]:

Backend:
- GET /api/v1/dashboard/stats (user-specific stats)

- GET /api/v1/dashboard/activity (recent activity)
- GET /api/v1/dashboard/summary

Frontend:
- Responsive dashboard layout
- Welcome header with user name
- Statistics cards:
  - Total items created
  - Active items
  - Recent activity count
  - Completion rate (if applicable)
- Recent activity timeline
- Quick action buttons
- Empty state for new users

Components:
- StatCard component (reusable)
- ActivityFeed component
- QuickActions panel
- Loading skeletons
- Personalized greeting based on time of day

17. Account Settings Page

Build comprehensive account settings for [Project Name]:

Settings Categories:

1. Profile Settings
   - Edit profile information

- Avatar management
- Bio and personal details

2. Security Settings
   - Change password form
   - Active sessions list
   - Login history

3. Notification Preferences
   - Email notifications (toggle)
   - Push notifications (toggle)
   - Notification frequency

4. Account Management
   - Export account data button
   - Delete account with confirmation

Backend:
- PATCH /api/v1/settings/profile
- PATCH /api/v1/settings/password
- PATCH /api/v1/settings/notifications
- GET /api/v1/settings/sessions
- POST /api/v1/settings/export-data
- DELETE /api/v1/users/me (account deletion)

Frontend:
- Tabbed or section-based layout
- Auto-save indicators
- Confirmation dialogs for critical actions
- Success/error notifications

18. Email Verification System

Implement email verification for [Project Name]:

Backend:

- Add fields to User: emailVerified, verificationToken, verificationExpires
- POST /api/v1/auth/verify-email/:token
  - Validate token
  - Check expiry (24 hours)
  - Mark email as verified
  - Return success

- POST /api/v1/auth/resend-verification
  - Generate new token
  - Send verification email
  - Rate limit (1 per minute)

Frontend:

- Email verification page (/verify-email/:token)
- Auto-verify on page load
- Success message with redirect
- Resend verification button
- Countdown timer (60 seconds)
- Banner on dashboard if unverified

Email Template:

- Professional verification email
- Clear call-to-action button
- Token expiry warning (24 hours)

19. Two-Factor Authentication (2FA)

Add optional 2FA functionality to [Project Name]:

Backend:

- Install 'speakeasy' and 'qrcode' packages
- Add to User schema: twoFactorSecret, twoFactorEnabled

Setup Flow:

- POST /api/v1/auth/2fa/setup
  - Generate secret
  - Generate QR code
  - Return QR code URL and manual entry key

- POST /api/v1/auth/2fa/verify-setup
  - Verify user's code
  - Enable 2FA if correct
  - Generate backup codes

- POST /api/v1/auth/2fa/disable
  - Verify code
  - Disable 2FA

Login with 2FA:

- Modify login flow to check if 2FA enabled
- Return 2FA required status
- POST /api/v1/auth/2fa/verify-login (verify code)

Frontend:

- 2FA setup wizard in settings
- QR code display
- Manual entry option

- Backup codes display and download
- 2FA toggle switch
- 6-digit code input on login

20. Session Management

Implement session tracking for [Project Name]:

Backend:

- Create sessions table/schema:
  - userId, token (hashed), device, browser
  - ipAddress, lastActivity, createdAt

- Store session on login
- GET /api/v1/sessions (list user's active sessions)
- DELETE /api/v1/sessions/:id (logout specific session)
- DELETE /api/v1/sessions/all (logout all devices)
- Update lastActivity on each authenticated request

Frontend:

- Active sessions page in settings
- Display: Device, Browser, Location, Last Active, Current Session indicator
- Logout button per session
- Logout all other devices button
- Confirmation dialog

Features:

- Detect device type (mobile/desktop/tablet)
- Parse user agent for browser/OS
- Show approximate location from IP
- Highlight current session

- Auto-expire inactive sessions (30 days)

21. Core App Data Schema

Design the main application schema for [Project Name]:

Schema: [Entity] (e.g., Task, Note, Post, Product)

MongoDB/Prisma Schema:

- id (auto-generated)
- userId (reference to User, indexed)
- title (required, string, max 200 chars)
- description (text, max 5000 chars)
- status (enum: draft, active, completed, archived)
- priority (enum: low, medium, high)
- tags (array of strings, optional)
- metadata (JSON, for flexible fields)
- isDeleted (boolean, for soft delete)
- createdAt, updatedAt (timestamps)

Relationships:

- belongsTo User
- Cascade on user delete (optional)

Indexes:

- userId (for user-specific queries)
- status (for filtering)
- createdAt (for sorting)

22. Create Operation (C in CRUD)

Implement CREATE functionality for [Entity] in [Project Name]:

Backend:

- POST /api/v1/[entities]
- Validate request body (title required, description optional)
- Check user authentication
- Set userId from authenticated user
- Create database record
- Return created entity with 201 status

Frontend:

- Create form page or modal
- Form fields: title, description, status, priority, tags
- Rich text editor for description (optional: Quill/TinyMCE)
- Real-time validation
- Submit button with loading state
- Success notification and redirect
- Form reset after creation
- Draft save functionality (optional)

Validation:

- Title: required, 3-200 characters
- Description: max 5000 characters
- Status: valid enum value
- Priority: valid enum value

23. Read Operations (R in CRUD)

Implement READ functionality for [Entity] in [Project Name]:

Backend:

- GET /api/v1/[entities] (list user's entities)
  - Filter by userId automatically
  - Support query params: page, limit, sort, order
  - Default: 20 items per page, sorted by createdAt desc
  - Return: data array + pagination metadata

- GET /api/v1/[entities]/:id (get single entity)
  - Check entity exists
  - Check user ownership
  - Return entity or 404

Frontend:
- List view page with cards or table
- Detail view page or modal
- Pagination controls
- Loading states (skeleton screens)
- Empty state ("No items yet")
- Refresh button

Response Format:

```
{
  success: true,
  data: [...],
  pagination: {
    page: 1,
    limit: 20,
    total: 150,
    totalPages: 8,
    hasNext: true,
    hasPrev: false
```

```
  }
}
```

24. Update Operation (U in CRUD)

Implement UPDATE functionality for [Entity] in [Project Name]:

Backend:

- PUT /api/v1/[entities]/:id (full update)
- PATCH /api/v1/[entities]/:id (partial update)
- Check entity exists
- Verify user ownership (userId match)
- Validate update data
- Update database record
- Return updated entity

Frontend:

- Edit form (pre-filled with current data)
- Same validation as create
- Unsaved changes warning
- Cancel/Save buttons
- Optimistic UI update
- Success notification
- Return to list or detail view

Features:

- Track dirty state (form changed)
- Inline editing for quick updates
- Auto-save draft (debounced)
- Compare changes (show what changed)

25. Delete Operation (D in CRUD)

Implement DELETE functionality for [Entity] in [Project Name]:

Backend:

- DELETE /api/v1/[entities]/:id
- Soft delete (recommended): set isDeleted: true, deletedAt: Date.now()
- OR Hard delete: actually remove from database
- Check entity exists
- Verify user ownership
- Return success message

Frontend:

- Delete button (trash icon)
- Confirmation modal:
  - Warning message
  - Entity title/name
  - "Are you sure?" prompt
  - Cancel/Delete buttons (Delete in red)
- Optimistic removal from UI
- Undo functionality (5 second window)
- Success notification
- Redirect if on detail page

Soft Delete Benefits:

- Data recovery possible
- Maintain audit trail
- Can be permanently deleted later
- Filter out deleted items in queries

26. Pagination Implementation

Add pagination to [Entity] list for [Project Name]:

Backend:

- Accept query params: page (default 1), limit (default 20, max 100)
- Calculate skip: (page - 1) * limit
- Query database with skip and limit
- Count total documents
- Return data + pagination metadata

Frontend:

- Pagination component at bottom of list
- Previous/Next buttons
- Page numbers (1 2 3 ... 10)
- Current page highlighted
- Items per page selector (10, 20, 50)
- Disable Previous on first page
- Disable Next on last page

URL Sync:

- Update URL with page parameter (?page=2)
- Read page from URL on mount
- Browser back/forward support

Alternative: Infinite scroll (see Prompt 57)

27. Search Functionality

Implement search for [Entity] in [Project Name]:

Backend:

- GET /api/v1/[entities]/search?q=query

- Text search on title and description
- MongoDB: Use text index and $text operator
- PostgreSQL: Use ILIKE or full-text search
- Filter by userId
- Return matching results

MongoDB Index:

```javascript
EntitySchema.index({ title: 'text', description: 'text' });
```

Frontend:

- Search input in list header
- Debounced search (300ms delay)
- Clear search button (X icon)
- Loading indicator while searching
- Show result count
- Empty state for no results
- Minimum 2 characters to search

Features:

- Highlight matching text in results
- Recent searches (localStorage)
- Search suggestions

28. Filtering System

Add filtering to [Entity] list in [Project Name]:

Backend:

- Accept filter query params: status, priority
- GET /api/v1/[entities]?status=active&priority=high

- Apply filters to database query
- Combine with pagination
- Return filtered results

Frontend:
- Filter dropdown/panel
- Checkboxes or radio buttons for:
  - Status (all, draft, active, completed)
  - Priority (all, low, medium, high)
- Apply filters button or auto-apply
- Clear filters button
- Active filter chips/badges
- Filter persistence in URL

URL Format:
/entities?page=1&status=active&priority=high&sort=createdAt&order=desc

Features:
- Multiple filters simultaneously
- Filter count indicator
- Saved filter presets (optional)

29. Sorting Mechanism
Implement sorting for [Entity] list in [Project Name]:

Backend:
- Accept sort query params: sort (field), order (asc/desc)
- GET /api/v1/[entities]?sort=createdAt&order=desc
- Validate sort field (whitelist)
- Apply sorting to database query

- Default sort: createdAt DESC

Sortable Fields:

- createdAt (default)
- updatedAt
- title (case-insensitive)
- priority
- status

Frontend:

- Sort dropdown OR clickable table headers
- Show current sort with icon (↑ ↓)
- Click to toggle asc/desc
- Clear sort (back to default)

Table View:

- Clickable column headers
- Sort icon indicator
- Visual feedback on active sort

Mobile View:

- Sort button with modal/drawer
- List of sort options

30. Data Validation

Implement comprehensive validation for [Project Name]:

Backend (using Zod or Joi):

- Create validation schemas for each entity
- Validate on CREATE and UPDATE

- Return clear validation errors

Example Zod Schema:

```typescript
const EntitySchema = z.object({
  title: z.string()
    .min(3, "Title must be at least 3 characters")
    .max(200, "Title too long"),
  description: z.string()
    .max(5000, "Description too long")
    .optional(),
  status: z.enum(['draft', 'active', 'completed']),
  priority: z.enum(['low', 'medium', 'high']),
  tags: z.array(z.string()).optional()
});
```

Frontend Validation:

- React Hook Form + Zod resolver
- Real-time validation (onBlur or onChange)
- Field-level error messages
- Form-level errors
- Disable submit if invalid

Error Display:

- Red border on invalid fields
- Error message below field
- Error summary at top (optional)
- Focus on first error field

Validation Types:

- Required fields
- String length (min/max)
- Enum values
- Array validation
- Custom business rules

31. File Upload System

Add file upload capability to [Project Name]:

Backend:

- Install 'multer' for file handling
- POST /api/v1/upload
- Configure storage: local OR cloud (Cloudinary recommended)
- File validation: type, size (max 10MB)
- Generate unique filename
- Save file metadata to database (if needed)
- Return file URL

Cloudinary Integration:

- Install 'cloudinary' package
- Configure with API credentials
- Upload files to Cloudinary
- Return secure URL

Frontend:

- File input with drag-and-drop
- Click to browse button
- File preview before upload
- Upload progress bar

- Multiple file support (optional)
- File size/type validation before upload
- Remove file button
- Success/error handling

Allowed File Types:
- Images: jpg, jpeg, png, gif, webp
- Documents: pdf
- Limit based on use case

Database (optional files table):
- id, userId, filename, originalName
- url, size, mimeType
- createdAt

32. Image Processing
Add image optimization to [Project Name]:

Backend:
- Install 'sharp' library
- Process uploaded images:
  - Resize to max width/height (e.g., 1200px)
  - Compress quality (80%)
  - Convert to WebP format
  - Generate thumbnail (200x200)

Image Sizes:
- Thumbnail: 200x200 (square crop)
- Medium: 800x800 (max dimensions)
- Large: 1200x1200 (max dimensions)

API:

- POST /api/v1/images/upload (upload and process)
- Return URLs for all sizes

Frontend:

- Image preview component
- Responsive images (use srcset)
- Lazy loading
- Fallback for unsupported browsers

Features:

- Avatar/profile picture upload
- Cover image upload
- Gallery image upload
- Automatic optimization

33. Real-Time Notifications

Implement in-app notifications for [Project Name]:

Backend:

- Create Notification schema:
  - userId, type, title, message
  - data (JSON), isRead, createdAt

- POST /api/v1/notifications (create notification)
- GET /api/v1/notifications (get user's notifications)
- PATCH /api/v1/notifications/:id/read (mark as read)
- PATCH /api/v1/notifications/read-all (mark all as read)
- DELETE /api/v1/notifications/:id (delete)

Notification Types:

- info, success, warning, error

Frontend:

- Notification bell icon in header
- Unread count badge
- Notification dropdown/panel
- List of notifications
- Mark as read on click
- Delete notification button
- Mark all as read button
- Toast notifications for new notifications

Features:

- Auto-refresh notifications
- Click notification to navigate
- Notification preferences
- Notification sound (optional)

34. WebSocket for Real-Time Updates

Set up WebSocket for real-time features in [Project Name]:

Backend (Socket.io):

- Install 'socket.io'
- Initialize Socket.io server
- Authentication middleware
- User room joining
- Event emitters

Server Setup:

```typescript
```

```
import { Server } from 'socket.io';

const io = new Server(httpServer, {
  cors: { origin: process.env.FRONTEND_URL }
});

io.on('connection', (socket) => {
  socket.on('join', (userId) => {
    socket.join(userId);
  });

  // Emit to specific user
  io.to(userId).emit('notification', data);
});
```

Frontend:
- Install 'socket.io-client'
- Connect on app initialization
- Join user room
- Listen for events
- Handle disconnection

Use Cases:
- Real-time notifications
- Live updates to entities
- Presence indicators
- Collaborative features

Events:

- notification (new notification)
- entity_created (new entity)
- entity_updated (entity changed)
- entity_deleted (entity removed)

35. Email Service Integration

Set up email service for [Project Name]:

Backend:
- Install 'nodemailer' OR use SendGrid/Mailgun
- Create email service module
- Email templates using Handlebars

Email Types:
1. Welcome email (on signup)
2. Email verification
3. Password reset
4. Notification emails

Nodemailer Setup:

```typescript
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: 587,
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASS
  }
});
```

```
async function sendEmail(to, subject, html) {
  await transporter.sendMail({
    from: process.env.FROM_EMAIL,
    to, subject, html
  });
}
```

Email Templates:
- HTML + plain text versions
- Responsive design
- Company branding
- Clear call-to-action buttons

Testing:
- Use Mailtrap for development
- Test all email templates

36. Third-Party API Integration

Integrate external APIs into [Project Name]:

Structure:
- Create /src/services/external folder
- One service file per API
- Centralized error handling
- API key management

Example: Weather API

```typescript
import axios from 'axios';
```

```
class WeatherService {
  private apiKey = process.env.WEATHER_API_KEY;
  private baseURL = 'https://api.weather.com';

  async getWeather(city: string) {
    const response = await axios.get(`${this.baseURL}/data`, {
      params: { city, apiKey: this.apiKey }
    });
    return response.data;
  }
}
```

Best Practices:
- Store API keys in environment variables
- Implement retry logic
- Cache responses (Redis/memory)
- Handle rate limits
- Validate API responses
- Proxy sensitive calls through backend
- Log API calls for debugging

Common Integrations:
- Payment: Stripe
- Maps: Google Maps
- Analytics: Google Analytics
- Email: SendGrid
- SMS: Twilio
- Storage: AWS S3, Cloudinary

37. Payment Integration (Stripe)

Integrate Stripe payments into [Project Name]:

Backend:

- Install 'stripe' SDK
- POST /api/v1/payments/create-intent
  - Create PaymentIntent
  - Return client_secret

- POST /api/v1/payments/webhook (Stripe events)
  - Verify webhook signature
  - Handle payment_intent.succeeded
  - Update order/subscription status

Database (transactions table):

- id, userId, stripePaymentId
- amount, currency, status
- metadata, createdAt

Frontend:

- Install '@stripe/stripe-js' '@stripe/react-stripe-js'
- Checkout page with Stripe Elements
- Card input component
- Submit payment button
- Loading and error states
- Success page

Payment Flow:

1. Create PaymentIntent (backend)

2. Show checkout form (frontend)

3. Collect card details

4. Confirm payment

5. Handle webhook (backend)

6. Show confirmation

Features:

- Multiple currencies
- Save cards for future
- Refund handling
- Receipt generation

38. Background Jobs (Optional)

Set up background job processing for [Project Name]:

Backend:

- Install 'bull' (Redis-based queue)
- Create job queues
- Job processors

Setup:

```typescript
import Bull from 'bull';

const emailQueue = new Bull('email', {
  redis: process.env.REDIS_URL
});

// Process emails
emailQueue.process(async (job) => {
  const { to, subject, html } = job.data;
```

```
  await sendEmail(to, subject, html);
});

// Add job
emailQueue.add({ to, subject, html }, {
  attempts: 3,
  backoff: 2000
});
```

Use Cases:
- Email sending (bulk or individual)
- Report generation
- Data export
- Image processing
- Cleanup tasks
- Scheduled jobs

Features:
- Retry failed jobs
- Job prioritization
- Delayed jobs
- Recurring jobs (cron)
- Job status tracking
- Admin dashboard (Bull Board)

39. Analytics & Tracking

Add analytics tracking to [Project Name]:

Backend:
- POST /api/v1/analytics/event

- Track user events:
  - Page views
  - Button clicks
  - Feature usage
  - Time spent

Event Schema:
```typescript
{
  userId,
  event: 'button_clicked',
  category: 'signup',
  properties: { button: 'start_trial' },
  timestamp: Date
}
```

Frontend:
- Create analytics hook
- Track page views automatically
- Track important actions

Google Analytics (optional):
- Install 'react-ga4'
- Initialize with tracking ID
- Track page views and events

Privacy:
- Respect user preferences
- Anonymous tracking option

- GDPR compliance

40. Export & Reporting

Add data export functionality to [Project Name]:

Backend:

- GET /api/v1/export/[entities]?format=csv
- Generate CSV from user's data
- Use 'fast-csv' or 'json2csv'
- Stream large datasets
- Return file for download

CSV Export:

```typescript
import { parse } from 'json2csv';

async function exportToCSV(data) {
  const csv = parse(data);
  return csv;
}
```

Frontend:

- Export button in list view
- Format selection (CSV, JSON)
- Download progress indicator
- Success notification

Features:

- Export all or filtered data
- Include/exclude columns

- Custom date range
- Email export link for large files

41. Frontend Component Library

Set up reusable UI component library for [Project Name]:

Install shadcn/ui:
- Button, Input, Label
- Card, Modal, Dropdown
- Table, Tabs, Toast
- Form components

Component Organization:

```
/components
  /ui (shadcn components)
  /common (reusable app components)
  /features (feature-specific)
  /layouts (page layouts)
```

Create Common Components:
- LoadingSpinner
- ErrorMessage
- EmptyState
- ConfirmDialog
- PageHeader
- DataTable

Best Practices:
- TypeScript interfaces for props
- Default props

- Error boundaries
- Accessibility (ARIA labels)

42. Loading & Skeleton States

Implement loading states for [Project Name]:

Frontend:
- Skeleton screens for lists
- Spinner for buttons during submission
- Progress bars for uploads
- Loading overlays for page transitions

Skeleton Components:

```tsx
function SkeletonCard() {
  return (
    <div className="animate-pulse">
      <div className="h-4 bg-gray-200 rounded w-3/4 mb-2"></div>
      <div className="h-3 bg-gray-200 rounded w-1/2"></div>
    </div>
  );
}
```

Loading States:
- Button disabled during API calls
- Show spinner on button
- Disable form inputs while submitting
- Page-level loading indicators
- Suspense boundaries (React 18)

Best Practices:

- Never block entire UI
- Show progress when possible
- Meaningful loading messages

43. Empty States & Fallbacks

Design empty states for [Project Name]:

Empty State Component:

```tsx
function EmptyState({ title, description, action }) {
  return (
    <div className="text-center py-12">
      <Icon />
      <h3>{title}</h3>
      <p>{description}</p>
      {action && <Button>{action}</Button>}
    </div>
  );
}
```

Empty States For:

- No entities created yet
- No search results found
- No notifications
- Filtered list with no results
- Error fallbacks

Features:

- Relevant illustration/icon

- Clear explanation
- Call-to-action button
- Helpful suggestions

44. Error Handling & User Feedback

Enhance error handling and feedback for [Project Name]:

Frontend:
- Toast notifications for actions
  - Success: green with checkmark
  - Error: red with X
  - Info: blue
  - Warning: yellow

- Inline form errors
- Error boundaries for React errors
- Global error handler for API errors

Toast Implementation:
- Install 'react-hot-toast' or 'sonner'
- Toast on successful operations
- Toast on errors
- Auto-dismiss after 3-5 seconds

Error Messages:
- User-friendly language
- Specific, not generic
- Actionable (suggest fix)
- Link to help docs if applicable

Axios Interceptor:

```typescript
axios.interceptors.response.use(
  response => response,
  error => {
    toast.error(error.response?.data?.message || 'Something went wrong');
    return Promise.reject(error);
  }
);
```

45. Responsive Design & Mobile

Optimize for mobile devices in [Project Name]:

Tailwind Responsive Classes:

- sm: (640px+)
- md: (768px+)
- lg: (1024px+)
- xl: (1280px+)

Mobile Considerations:

- Touch-friendly buttons (min 44x44px)
- Readable font sizes (16px+ for body)
- Adequate spacing
- Hamburger menu for navigation
- Bottom navigation for mobile
- Swipe gestures
- Pull to refresh

Responsive Patterns:

- Stack columns on mobile
- Hide/show elements based on screen size

- Collapsible sections
- Modal instead of sidebar on mobile
- Responsive tables (horizontal scroll or stacked)

Testing:
- Test on multiple device sizes
- Chrome DevTools device emulation
- Real device testing

46. Dark Mode Implementation

Add dark mode to [Project Name]:

Tailwind Dark Mode:
- Configure in tailwind.config.js
- Use 'class' strategy
- Apply dark: prefix to classes

Theme Context:

```tsx
const ThemeContext = createContext();

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  useEffect(() => {
    const saved = localStorage.getItem('theme') || 'light';
    setTheme(saved);
    document.documentElement.classList.toggle('dark', saved === 'dark');
  }, []);

  const toggleTheme = () => {
```

```
    const newTheme = theme === 'light' ? 'dark' : 'light';
    setTheme(newTheme);
    localStorage.setItem('theme', newTheme);
    document.documentElement.classList.toggle('dark');
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Dark Mode Styles:

```css
/* Light mode (default) */
body {
  @apply bg-white text-gray-900;
}

/* Dark mode */
.dark body {
  @apply bg-gray-900 text-white;
}
```

Features:

- Toggle switch in header/settings
- System preference detection

- Smooth transitions
- All components styled for dark mode

47. Performance Optimization

Optimize performance for [Project Name]:

Frontend:
- Code splitting (lazy loading routes)
- Image optimization (next/image or lazy loading)
- Debounce search inputs
- Virtualize long lists (react-window)
- Memoize expensive calculations
- Remove unused dependencies

Backend:
- Database query optimization
  - Add indexes
  - Select only needed fields
  - Use lean() for read-only queries
- Cache frequently accessed data (Redis)
- Compress responses (gzip)
- Rate limiting
- Connection pooling

Bundle Size:
- Analyze bundle (webpack-bundle-analyzer)
- Remove unused code
- Use tree-shaking
- Lazy load heavy libraries

Caching:

- HTTP caching headers
- Browser caching (service workers)
- API response caching
- Static asset CDN

48. SEO & Meta Tags

Implement SEO best practices for [Project Name]:

Frontend (Next.js):
- Dynamic meta tags per page
- Open Graph tags
- Twitter Card tags
- Canonical URLs
- Sitemap.xml
- Robots.txt

Meta Tags:

```tsx
import Head from 'next/head';

function Page() {
  return (
    <>
      <Head>
        <title>Page Title | App Name</title>
        <meta name="description" content="Page description" />
        <meta property="og:title" content="Page Title" />
        <meta property="og:description" content="Description" />
        <meta property="og:image" content="/og-image.jpg" />
        <meta name="twitter:card" content="summary_large_image" />
      </Head>
```

```
      {/* Page content */}
    </>
  );
}
```

Best Practices:

- Unique title per page (50-60 chars)
- Descriptive meta descriptions (150-160 chars)
- Semantic HTML (h1, h2, etc.)
- Alt text for images
- Structured data (JSON-LD)

49. Security Hardening

Enhance security for [Project Name]:

Backend:

- Helmet.js (security headers)
- CORS configuration
- Rate limiting (express-rate-limit)
- Input sanitization
- SQL injection prevention (use ORMs)
- XSS prevention
- CSRF tokens (if using cookies)
- Validate all inputs
- Escape outputs

Environment:

- Never commit .env files
- Use secrets management (production)
- Rotate API keys regularly

- Minimum privilege principle

Frontend:
- Sanitize user input before display
- HttpOnly cookies for tokens
- HTTPS only
- Content Security Policy
- Validate data from backend

Security Headers:
```typescript
app.use(helmet({
  contentSecurityPolicy: true,
  xssFilter: true,
  noSniff: true,
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' }
}));
```

Testing:
- Penetration testing
- Security audit
- Dependency vulnerability scanning

50. Testing Setup

Set up testing for [Project Name]:

Backend Testing (Jest):
- Unit tests for services
- Integration tests for APIs
- Test database (separate from dev)

Example:

```typescript
describe('Auth API', () => {
  it('should register a user', async () => {
    const response = await request(app)
      .post('/api/v1/auth/signup')
      .send({ email: 'test@example.com', password: 'Test123!' });

    expect(response.status).toBe(201);
    expect(response.body.success).toBe(true);
  });
});
```

Frontend Testing:

- Component tests (React Testing Library)
- Integration tests
- E2E tests (Playwright/Cypress)

Test Coverage:

- Aim for 70%+ coverage
- Test critical paths
- Test error scenarios

CI/CD Integration:

- Run tests on every commit
- Block deployment if tests fail

51. Environment Separation

Configure separate environments for [Project Name]:

Environments:

1. Development (local)
2. Staging (pre-production)
3. Production

Environment Variables:

- Separate .env files
- .env.development
- .env.staging
- .env.production

Backend:

- Different database URLs
- Different API keys
- Different CORS origins
- Different log levels

Frontend:

- Different API URLs
- Different analytics IDs
- Different feature flags

Deployment:

- Use environment variables in hosting platform
- Never commit production secrets
- Use CI/CD to inject env vars

52. Docker Setup

Dockerize [Project Name]:

Dockerfile (Backend):

```dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build
EXPOSE 5000
CMD ["node", "dist/server.js"]
```

Dockerfile (Frontend):

```dockerfile
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Docker Compose:

```yaml
```

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=${DATABASE_URL}

  frontend:
    build: ./frontend
    ports:
      - "3000:80"
```

Benefits:
- Consistent environments
- Easy deployment
- Scalability

53. Backend Deployment

Deploy backend to production for [Project Name]:

Platform Options:
1. Railway (easiest)
2. Render
3. Fly.io
4. AWS (advanced)
5. DigitalOcean

Railway Deployment:

1. Connect GitHub repo
2. Add environment variables
3. Deploy automatically
4. Get deployment URL

Requirements:
- package.json with start script
- Environment variables configured
- Database connection URL
- PORT from environment

Post-Deployment:
- Test all API endpoints
- Monitor logs
- Set up alerts
- Configure custom domain (optional)

Scaling:
- Auto-scaling configuration
- Load balancing
- Health checks

54. Frontend Deployment

Deploy frontend to production for [Project Name]:

Vercel Deployment (Recommended):
1. Import GitHub repo
2. Configure build settings:
   - Framework: Next.js/React
   - Build command: npm run build
   - Output directory: dist or .next

3. Add environment variables

4. Deploy

Environment Variables:

- VITE_API_URL (backend URL)
- Next: NEXT_PUBLIC_API_URL

Post-Deployment:

- Test all pages
- Check API connectivity
- Test authentication flow
- Verify production API URL

Custom Domain:

- Add domain in Vercel
- Configure DNS records
- Enable HTTPS (automatic)
- Redirect www to non-www (or vice versa)

Performance:

- Enable Vercel Analytics
- Configure caching
- Optimize images

55. Database Migration

Set up database migrations for [Project Name]:

MongoDB (mongoose):

- Use migration tool (migrate-mongo)
- Create migration files
- Up/down functions

- Version control migrations

PostgreSQL (Prisma):

```bash
# Create migration
npx prisma migrate dev --name init

# Apply to production
npx prisma migrate deploy
```

Migration Strategy:

- Always test migrations locally first
- Backup database before production migration
- Run migrations before deploying code
- Never modify old migrations
- Document breaking changes

Rollback Plan:

- Keep migration history
- Test rollback procedures
- Backup before major changes

56. CI/CD Pipeline

Set up CI/CD for [Project Name]:

GitHub Actions:

```yaml
name: Deploy

on:
```

```
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
      - run: npm ci
      - run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to production
        run: |
          # Deployment commands
```

Pipeline Steps:

1. Run tests
2. Run linting
3. Build application
4. Deploy if all pass
5. Notify on failure

Benefits:

- Automated deployments

- Catch bugs before production
- Consistent deployment process

57. Monitoring & Logging

Set up monitoring for [Project Name]:

Application Monitoring:
- Install Sentry for error tracking
- Track exceptions and errors
- User impact metrics
- Performance monitoring

Backend Logging:
- Structured logging (Winston/Pino)
- Log levels: error, warn, info, debug
- Log important events
- Don't log sensitive data

Frontend Monitoring:
- Error boundaries
- Track JavaScript errors
- Performance metrics (Web Vitals)

Alerts:
- Email/Slack on critical errors
- Database connection failures
- API errors above threshold
- Server resource alerts

Metrics to Track:
- Response times

- Error rates
- User activity
- Database query performance

58. Backup Strategy

Implement backup system for [Project Name]:

Database Backups:
- Automated daily backups
- Retention: 30 days
- Test restore procedure
- Store backups securely

MongoDB Atlas:
- Automatic backups enabled
- Point-in-time recovery
- Download backup snapshots

PostgreSQL:
- pg_dump for backups
- Automated with cron
- Store in S3 or similar

File Backups:
- User uploaded files
- Application logs
- Configuration files

Testing:
- Test restore monthly
- Document restore procedure

- Practice disaster recovery

59. SSL/HTTPS Configuration

Configure HTTPS for [Project Name]:

Frontend (Vercel/Netlify):

- Automatic HTTPS
- SSL certificate auto-renewal
- Force HTTPS redirect

Backend (Railway/Render):

- Automatic HTTPS
- Custom domain with SSL

Custom Server:

- Use Let's Encrypt (free SSL)
- Certbot for certificate management
- Auto-renewal setup

nginx Configuration:

```nginx
server {
  listen 443 ssl;
  ssl_certificate /path/to/cert.pem;
  ssl_certificate_key /path/to/key.pem;

  location / {
    proxy_pass http://localhost:5000;
  }
}
```

Security:

- Force HTTPS (redirect HTTP to HTTPS)
- HSTS header
- Secure cookies (secure flag)

60. Production Checklist

Pre-launch checklist for [Project Name]:

Security:

- [ ] All API routes protected
- [ ] Input validation on all endpoints
- [ ] Environment variables secured
- [ ] HTTPS enabled
- [ ] CORS configured correctly
- [ ] Rate limiting active
- [ ] Security headers configured

Performance:

- [ ] Database indexes created
- [ ] API response times < 500ms
- [ ] Images optimized
- [ ] Caching implemented
- [ ] Bundle size optimized

Functionality:

- [ ] All features tested
- [ ] Authentication flow works
- [ ] Password reset works
- [ ] Email sending works
- [ ] File uploads work

- [ ] Payments work (if applicable)

Monitoring:

- [ ] Error tracking active
- [ ] Logging configured
- [ ] Alerts set up
- [ ] Backups scheduled
- [ ] Analytics tracking

Documentation:

- [ ] API documentation
- [ ] README updated
- [ ] Environment variables documented
- [ ] Deployment process documented

61. Advanced Data Visualization

Create interactive visualizations for [Project Name]:

Install Recharts:

```bash
npm install recharts
```

Chart Types:

1. Line Chart (Trends):

```tsx
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from 'recharts';

function TrendChart({ data }) {
```

```
  return (
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={data}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="date" />
        <YAxis />
        <Tooltip />
        <Legend />
        <Line type="monotone" dataKey="value" stroke="#8884d8" />
      </LineChart>
    </ResponsiveContainer>
  );
}
```

2. Bar Chart (Comparisons):
3. Pie Chart (Distribution):
4. Area Chart (Cumulative):

Features:
- Interactive tooltips
- Responsive design
- Legend toggles
- Zoom capabilities
- Export as image
- Loading states
- Empty states

Use Cases:
- User growth over time

- Activity by day/week/month
- Category distribution
- Performance metrics

62. Keyboard Shortcuts

Implement keyboard shortcuts for [Project Name]:

Install react-hotkeys-hook:

```bash
npm install react-hotkeys-hook
```

Common Shortcuts:

```tsx
import { useHotkeys } from 'react-hotkeys-hook';

function App() {
  // Global search
  useHotkeys('ctrl+k, cmd+k', (e) => {
    e.preventDefault();
    openSearch();
  });

  // Save
  useHotkeys('ctrl+s, cmd+s', (e) => {
    e.preventDefault();
    handleSave();
  });

  // New item
  useHotkeys('ctrl+n, cmd+n', (e) => {
```

```
    e.preventDefault();
    createNew();
  });

  // Close modal
  useHotkeys('esc', () => closeModal());
}
```

Shortcuts to Implement:
- Ctrl/Cmd + K: Global search
- Ctrl/Cmd + S: Save
- Ctrl/Cmd + N: New item
- Escape: Close modal/cancel
- ?: Show shortcuts help

Shortcuts Modal:
- Display all available shortcuts
- Organized by category
- Keyboard icons for visual clarity

63. Infinite Scroll

Add infinite scroll pagination for [Project Name]:

Install library:
```bash
npm install react-infinite-scroll-component
```

Implementation:
```tsx
```

```
import InfiniteScroll from 'react-infinite-scroll-component';

function InfiniteList() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);

  const fetchMore = async () => {
    const newItems = await api.get(`/entities?page=${page}&limit=20`);

    if (newItems.length === 0) {
      setHasMore(false);
      return;
    }

    setItems(prev => [...prev, ...newItems]);
    setPage(prev => prev + 1);
  };

  return (
    <InfiniteScroll
      dataLength={items.length}
      next={fetchMore}
      hasMore={hasMore}
      loader={<LoadingSpinner />}
      endMessage={<p>No more items</p>}
    >
      {items.map(item => <ItemCard key={item.id} item={item} />)}
    </InfiniteScroll>
  );
```

}
```

Features:
- Automatic loading on scroll
- Loading indicator
- End message
- Pull to refresh (mobile)
- Fallback "Load More" button

64. Drag and Drop

Implement drag-and-drop for [Project Name]:

Install library:
```bash
npm install @dnd-kit/core @dnd-kit/sortable
```

Use Cases:
- Reorder list items
- Kanban board
- File upload
- Dashboard widgets

Sortable List Example:
```tsx
import { DndContext, closestCenter } from '@dnd-kit/core';
import { SortableContext, verticalListSortingStrategy, useSortable } from '@dnd-kit/sortable';

function SortableList({ items, onReorder }) {
  function handleDragEnd(event) {

```
    const { active, over } = event;
    if (active.id !== over.id) {
      // Update order
      onReorder(active.id, over.id);
    }
  }

  return (
    <DndContext collisionDetection={closestCenter} onDragEnd={handleDragEnd}>
      <SortableContext items={items} strategy={verticalListSortingStrategy}>
        {items.map(item => <SortableItem key={item.id} item={item} />)}
      </SortableContext>
    </DndContext>
  );
}
```

Features:

- Visual feedback while dragging
- Smooth animations
- Touch support (mobile)
- Keyboard accessibility
- Update backend on drop

65. Advanced Search with Filters

Create advanced search with filters for [Project Name]:

Backend:

- GET /api/v1/search?q=query&filter[status]=active&filter[priority]=high
- Full-text search
- Multiple filter support

- Faceted search (counts per filter)

Frontend:

Instant Search:

```tsx
function AdvancedSearch() {
  const [query, setQuery] = useState('');
  const [filters, setFilters] = useState({});
  const [results, setResults] = useState([]);

  useEffect(() => {
    const timer = setTimeout(async () => {
      if (query.length >= 2) {
        const data = await searchAPI(query, filters);
        setResults(data);
      }
    }, 300); // Debounce

    return () => clearTimeout(timer);
  }, [query, filters]);

  return (
    <div>
      <SearchInput value={query} onChange={e => setQuery(e.target.value)} />
      <FilterPanel filters={filters} onChange={setFilters} />
      <SearchResults results={results} />
    </div>
  );
}
```

```

Filter Panel:
- Checkboxes for categories
- Range sliders for numbers
- Date range pickers
- Multi-select dropdowns

Features:
- Search suggestions
- Recent searches
- Save search filters
- Share search URL
- Highlight matching text
66. Real-Time Collaboration
Add collaborative features to [Project Name]:

Backend (Socket.io):
- Track online users
- Broadcast changes to room
- Handle concurrent edits

User Presence:
```tsx
function OnlineUsers() {
  const [onlineUsers, setOnlineUsers] = useState([]);

  useEffect(() => {
    socket.emit('join_presence');
```
```

```
    socket.on('users_online', setOnlineUsers);
    socket.on('user_joined', (user) => {
      setOnlineUsers(prev => [...prev, user]);
      toast.info(`${user.name} joined`);
    });
    socket.on('user_left', (userId) => {
      setOnlineUsers(prev => prev.filter(u => u.id !== userId));
    });

    return () => {
      socket.off('users_online');
      socket.off('user_joined');
      socket.off('user_left');
    };
  }, []);

  return (
    <div>
      <h3>Online ({onlineUsers.length})</h3>
      {onlineUsers.map(user => (
        <UserAvatar key={user.id} user={user} online />
      ))}
    </div>
  );
}
```

Features:

- Online presence indicators
- Typing indicators

- Live cursor positions (advanced)
- Real-time updates to shared content
- Conflict resolution

67. Activity Feed & Timeline

Create activity feed for [Project Name]:

Backend:
- GET /api/v1/activity (user's activity)
- GET /api/v1/activity/team (team activity)
- Return chronological list of activities

Activity Types:
- created, updated, deleted
- status_changed, assigned
- commented, shared

Frontend:

```tsx
function ActivityFeed() {
  const [activities, setActivities] = useState([]);

  useEffect(() => {
    fetchActivities().then(setActivities);
  }, []);

  return (
    <div className="space-y-4">
      {activities.map(activity => (
        <ActivityItem key={activity.id}>
          <UserAvatar user={activity.user} />
```

```
          <div>
            <ActivityText activity={activity} />
            <TimeAgo date={activity.createdAt} />
          </div>
        </ActivityItem>
      ))}
    </div>
  );
}

function ActivityText({ activity }) {
  const templates = {
    created: `created ${activity.entityType} "${activity.entityTitle}"`,
    updated: `updated ${activity.entityType} "${activity.entityTitle}"`,
    deleted: `deleted ${activity.entityType}`,
    status_changed: `changed status to ${activity.newStatus}`
  };

  return <p>{activity.user.name} {templates[activity.action]}</p>;
}
```

Features:

- Group by date
- Filter by activity type
- Infinite scroll
- Real-time updates
- Click to view entity

68. Theming System

Implement advanced theming for [Project Name]:

Theme Configuration:

```tsx
const themes = {
  light: {
    primary: '#3B82F6',
    secondary: '#10B981',
    background: '#FFFFFF',
    foreground: '#1F2937',
    border: '#E5E7EB',
    accent: '#F59E0B'
  },
  dark: {
    primary: '#60A5FA',
    secondary: '#34D399',
    background: '#1F2937',
    foreground: '#F9FAFB',
    border: '#374151',
    accent: '#FBBF24'
  },
  blue: {
    // Custom theme
  }
};

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  useEffect(() => {
    const root = document.documentElement;
```

```
    const colors = themes[theme];

    Object.entries(colors).forEach(([key, value]) => {
      root.style.setProperty(`--color-${key}`, value);
    });
  }, [theme]);

  return (
    <ThemeContext.Provider value={{ theme, setTheme, themes }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Features:
- Multiple theme options
- Custom theme colors
- User preference saved
- Smooth transitions
- Theme picker in settings

69. Accessibility (A11y)

Ensure accessibility for [Project Name]:

ARIA Labels:

```tsx
<button aria-label="Close modal" onClick={close}>
  <X />
</button>
```

```
<input
  type="text"
  aria-label="Search items"
  aligned-describedby="search-help"
/>
<span id="search-help">Enter at least 2 characters</span>
```

Keyboard Navigation:
- Tab through all interactive elements
- Enter to activate buttons
- Escape to close modals
- Arrow keys for navigation

Focus Management:
- Visible focus indicators
- Trap focus in modals
- Return focus after modal closes
- Skip to main content link

Screen Readers:
- Semantic HTML (header, nav, main, footer)
- alt text for images
- Descriptive link text
- ARIA live regions for dynamic content

Color Contrast:
- Minimum 4.5:1 for normal text
- Minimum 3:1 for large text
- Don't rely on color alone

Testing:

- Test with keyboard only
- Use screen reader (NVDA/JAWS)
- Automated tools (axe DevTools)

70. Internationalization (i18n)

Add multi-language support to [Project Name]:

Install i18next:

```bash
npm install i18next react-i18next
```

Setup:

```tsx
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';

i18n.use(initReactI18next).init({
  resources: {
    en: {
      translation: {
        welcome: "Welcome",
        login: "Login",
        signup: "Sign Up"
      }
    },
    es: {
      translation: {
        welcome: "Bienvenido",
```

```
        login: "Iniciar sesión",
        signup: "Registrarse"
      }
    }
  },
  lng: 'en',
  fallbackLng: 'en'
});
```

Usage:

```tsx
import { useTranslation } from 'react-i18next';

function Component() {
  const { t, i18n } = useTranslation();

  return (
    <div>
      <h1>{t('welcome')}</h1>
      <button onClick={() => i18n.changeLanguage('es')}>
        Español
      </button>
    </div>
  );
}
```

Features:

- Language selector

- Save preference
- Date/number formatting
- RTL support (Arabic, Hebrew)
- Dynamic content translation

71. Unit Testing - Backend

Write unit tests for backend services in [Project Name]:

Setup Jest:

```bash
npm install --save-dev jest @types/jest ts-jest supertest
```

Service Tests:

```typescript
import { AuthService } from './auth.service';

describe('AuthService', () => {
  let authService: AuthService;

  beforeEach(() => {
    authService = new AuthService();
  });

  describe('hashPassword', () => {
    it('should hash password correctly', async () => {
      const password = 'Test123!';
      const hashed = await authService.hashPassword(password);

      expect(hashed).not.toBe(password);
```

```
    expect(hashed).toHaveLength(60); // bcrypt hash length
  });
});

describe('comparePasswords', () => {
  it('should return true for matching passwords', async () => {
    const password = 'Test123!';
    const hashed = await authService.hashPassword(password);
    const isMatch = await authService.comparePasswords(password, hashed);

    expect(isMatch).toBe(true);
  });

  it('should return false for non-matching passwords', async () => {
    const hashed = await authService.hashPassword('Test123!');
    const isMatch = await authService.comparePasswords('Wrong123!', hashed);

    expect(isMatch).toBe(false);
  });
});
});
```

Test Coverage Goals:

- Services: 80%+
- Utils: 90%+
- Critical paths: 100%

72. Integration Testing - API

Write API integration tests for [Project Name]:

Setup:

```typescript
import request from 'supertest';
import app from '../app';

describe('Auth API', () => {
  describe('POST /api/v1/auth/signup', () => {
    it('should register a new user', async () => {
      const response = await request(app)
        .post('/api/v1/auth/signup')
        .send({
          name: 'Test User',
          email: 'test@example.com',
          password: 'Test123!'
        });

      expect(response.status).toBe(201);
      expect(response.body.success).toBe(true);
      expect(response.body.data.user).toHaveProperty('id');
      expect(response.body.data).toHaveProperty('token');
    });

    it('should reject duplicate email', async () => {
      // First signup
      await request(app).post('/api/v1/auth/signup').send({
        name: 'Test',
        email: 'duplicate@example.com',
        password: 'Test123!'
      });
```

```
    // Duplicate signup
    const response = await request(app).post('/api/v1/auth/signup').send({
      name: 'Test2',
      email: 'duplicate@example.com',
      password: 'Test123!'
    });

    expect(response.status).toBe(400);
    expect(response.body.success).toBe(false);
  });

  it('should reject weak password', async () => {
    const response = await request(app).post('/api/v1/auth/signup').send({
      name: 'Test',
      email: 'test2@example.com',
      password: 'weak'
    });

    expect(response.status).toBe(400);
  });
});

describe('POST /api/v1/auth/login', () => {
  beforeEach(async () => {
    // Create test user
    await request(app).post('/api/v1/auth/signup').send({
      name: 'Login Test',
      email: 'login@example.com',
      password: 'Test123!'
    });
```

```
    });

    it('should login with correct credentials', async () => {
      const response = await request(app).post('/api/v1/auth/login').send({
        email: 'login@example.com',
        password: 'Test123!'
      });

      expect(response.status).toBe(200);
      expect(response.body.data).toHaveProperty('token');
    });

    it('should reject incorrect password', async () => {
      const response = await request(app).post('/api/v1/auth/login').send({
        email: 'login@example.com',
        password: 'WrongPassword123!'
      });

      expect(response.status).toBe(401);
    });
  });
});
```

Test Database:

- Use separate test database
- Clear database before each test suite
- Seed test data as needed

73. Frontend Component Testing

Write component tests for [Project Name]:

Setup React Testing Library:

```bash
npm install --save-dev @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

Component Tests:

```tsx
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import LoginForm from './LoginForm';

describe('LoginForm', () => {
  it('renders login form', () => {
    render(<LoginForm />);

    expect(screen.getByLabelText(/email/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/password/i)).toBeInTheDocument();
    expect(screen.getByRole('button', { name: /login/i })).toBeInTheDocument();
  });

  it('shows validation errors for empty fields', async () => {
    render(<LoginForm />);

    const submitButton = screen.getByRole('button', { name: /login/i });
    fireEvent.click(submitButton);

    await waitFor(() => {
      expect(screen.getByText(/email is required/i)).toBeInTheDocument();
      expect(screen.getByText(/password is required/i)).toBeInTheDocument();
```

```
  });
});

it('submits form with valid data', async () => {
  const mockLogin = jest.fn();
  render(<LoginForm onLogin={mockLogin} />);

  await userEvent.type(screen.getByLabelText(/email/i), 'test@example.com');
  await userEvent.type(screen.getByLabelText(/password/i), 'Test123!');
  fireEvent.click(screen.getByRole('button', { name: /login/i }));

  await waitFor(() => {
    expect(mockLogin).toHaveBeenCalledWith({
      email: 'test@example.com',
      password: 'Test123!'
    });
  });
});

it('displays error message on login failure', async () => {
  const mockLogin = jest.fn().mockRejectedValue(new Error('Invalid credentials'));
  render(<LoginForm onLogin={mockLogin} />);

  await userEvent.type(screen.getByLabelText(/email/i), 'test@example.com');
  await userEvent.type(screen.getByLabelText(/password/i), 'wrong');
  fireEvent.click(screen.getByRole('button', { name: /login/i }));

  await waitFor(() => {
    expect(screen.getByText(/invalid credentials/i)).toBeInTheDocument();
  });
```

```
  });
});
```

Testing Best Practices:

- Test user behavior, not implementation
- Use accessibility queries (getByRole, getByLabelText)
- Test error states
- Test loading states
- Mock API calls

74. E2E Testing

Write end-to-end tests for [Project Name]:

Setup Playwright:

```bash
npm install --save-dev @playwright/test
npx playwright install
```

E2E Tests:

```typescript
import { test, expect } from '@playwright/test';

test.describe('User Authentication Flow', () => {
  test('should complete signup flow', async ({ page }) => {
    await page.goto('http://localhost:3000/signup');

    // Fill signup form
    await page.fill('[name="name"]', 'Test User');
    await page.fill('[name="email"]', 'testuser@example.com');
```

```
  await page.fill('[name="password"]', 'Test123!');
  await page.fill('[name="confirmPassword"]', 'Test123!');

  // Submit
  await page.click('button[type="submit"]');

  // Should redirect to dashboard
  await expect(page).toHaveURL(/.*dashboard/);
  await expect(page.locator('h1')).toContainText('Dashboard');
});

test('should complete login flow', async ({ page }) => {
  await page.goto('http://localhost:3000/login');

  await page.fill('[name="email"]', 'testuser@example.com');
  await page.fill('[name="password"]', 'Test123!');
  await page.click('button[type="submit"]');

  await expect(page).toHaveURL(/.*dashboard/);
});

test('should create new item', async ({ page }) => {
  // Login first
  await page.goto('http://localhost:3000/login');
  await page.fill('[name="email"]', 'testuser@example.com');
  await page.fill('[name="password"]', 'Test123!');
  await page.click('button[type="submit"]');

  // Navigate to create page
  await page.click('text=New Item');
```

```
    // Fill form
    await page.fill('[name="title"]', 'Test Item');
    await page.fill('[name="description"]', 'This is a test item');
    await page.selectOption('[name="status"]', 'active');

    // Submit
    await page.click('button[type="submit"]');

    // Should see success message
    await expect(page.locator('.toast')).toContainText('Item created');

    // Should appear in list
    await page.click('text=Items');
    await expect(page.locator('text=Test Item')).toBeVisible();
  });
});
```

CI Integration:
- Run E2E tests in CI pipeline
- Use headless mode
- Screenshot on failure

75. Performance Testing

Test performance of [Project Name]:

Backend Load Testing (Artillery):

```bash
npm install --save-dev artillery
```

Load Test Configuration:

```yaml
# load-test.yml
config:
  target: "http://localhost:5000"
  phases:
    - duration: 60
      arrivalRate: 10  # 10 requests per second

scenarios:
  - name: "API Load Test"
    flow:
      - get:
          url: "/api/v1/entities"
          headers:
            Authorization: "Bearer {{ token }}"
      - post:
          url: "/api/v1/entities"
          json:
            title: "Load Test Item"
            description: "Testing load"
```

Frontend Performance:

```tsx
import { measurePerformance } from './utils/performance';

// Measure component render time
const PerformanceWrapper = ({ children }) => {
```

```
  useEffect(() => {
    const start = performance.now();
    return () => {
      const duration = performance.now() - start;
      console.log(`Component rendered in ${duration}ms`);
    };
  }, []);

  return children;
};
```

Lighthouse CI:
- Automate Lighthouse audits
- Set performance budgets
- Block deployment if scores drop

Metrics to Track:
- API response time (p95 < 500ms)
- Time to First Byte (< 200ms)
- First Contentful Paint (< 1.8s)
- Time to Interactive (< 3.8s)

76. Security Testing

Perform security testing for [Project Name]:

SQL Injection Testing:
- Try malicious SQL in inputs
- Verify parameterized queries prevent injection

XSS Testing:

- Input script tags
- Verify output is escaped

CSRF Testing:
- Verify CSRF tokens required
- Test without tokens

Authentication Testing:
- Try accessing protected routes without token
- Try with expired token
- Try with modified token

Authorization Testing:
- Try accessing other users' data
- Try admin routes as regular user

Rate Limiting Testing:
- Send rapid requests
- Verify 429 responses

Security Headers:
```bash
curl -I https://yourapp.com

# Should see:
# X-Content-Type-Options: nosniff
# X-Frame-Options: DENY
# X-XSS-Protection: 1; mode=block
# Strict-Transport-Security: max-age=31536000
```

Dependency Scanning:

```bash
npm audit
npm audit fix
```

OWASP Top 10:

- Test for all OWASP vulnerabilities
- Use automated scanners
- Manual penetration testing

77. Code Quality & Linting

Enforce code quality for [Project Name]:

ESLint Configuration:

```json
{
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:react/recommended",
    "plugin:react-hooks/recommended"
  ],
  "rules": {
    "no-console": "warn",
    "no-unused-vars": "error",
    "@typescript-eslint/no-explicit-any": "warn",
    "react/prop-types": "off"
  }
}
```

```

Prettier Configuration:
```json
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 80
}
```

Husky Pre-commit Hooks:
```bash
npm install --save-dev husky lint-staged
npx husky-init
```
```json
{
  "lint-staged": {
    "*.{js,jsx,ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{json,md}": ["prettier --write"]
  }
}
```

SonarQube (Optional):
- Code quality metrics
- Security hotspots
```

- Code smells
- Technical debt

Code Review Checklist:
- [ ] No console.logs in production
- [ ] Error handling present
- [ ] Tests written
- [ ] TypeScript types defined
- [ ] Comments for complex logic
- [ ] No hardcoded values

78. Documentation

Create comprehensive documentation for [Project Name]:

README.md:

```markdown
# Project Name

## Description
Brief description of the project

## Features
- Feature 1
- Feature 2
- Feature 3

## Tech Stack
- Backend: Node.js, Express, MongoDB
- Frontend: React, TypeScript, Tailwind CSS
- Deployment: Vercel, Railway
```

Setup Instructions

Prerequisites

- Node.js 18+
- MongoDB
- npm or yarn

Installation

\\```bash

Clone repository

git clone https://github.com/username/project.git

Install backend dependencies

cd backend

npm install

Install frontend dependencies

cd frontend

npm install

\\```

Environment Variables

Create `.env` files based on `.env.example`

Running Locally

\\```bash

Backend

cd backend

npm run dev

Frontend
cd frontend
npm run dev
\\```

API Documentation
See [API.md](./API.md)

Contributing
See [CONTRIBUTING.md](./CONTRIBUTING.md)

License
MIT
```

API Documentation (API.md):
- List all endpoints
- Request/response examples
- Authentication requirements
- Error responses

Code Comments:
- Document complex functions
- Explain business logic
- Document edge cases
- Keep comments up to date

Changelog:
- Document version changes
- Breaking changes
```

- New features
- Bug fixes

79. Continuous Improvement

Implement continuous improvement process for [Project Name]:

Monitoring & Metrics:
- Track key performance indicators
- User feedback collection
- Error rate monitoring
- Performance metrics

User Feedback:
- In-app feedback widget
- User surveys
- Feature requests
- Bug reports

Analytics:
- User behavior tracking
- Feature usage statistics
- Conversion funnels
- Retention metrics

Regular Reviews:
- Weekly: Bug review
- Bi-weekly: Performance review
- Monthly: Feature review
- Quarterly: Architecture review

Improvement Backlog:

- Prioritize by impact
- Estimate effort
- Plan sprints
- Track progress

A/B Testing:
- Test new features
- Compare designs
- Measure impact
- Roll out winners

80. Maintenance Plan

Create maintenance plan for [Project Name]:

Regular Tasks:

Daily:
- Monitor error logs
- Check server health
- Review user feedback

Weekly:
- Dependency updates (patch versions)
- Database backups verification
- Performance metrics review

Monthly:
- Dependency updates (minor versions)
- Security audit
- Database optimization
- Cost review

Quarterly:

- Major dependency updates
- Feature review
- Architecture review
- Security penetration testing

Annually:

- Technology stack review
- Infrastructure review
- Disaster recovery testing
- Legal/compliance review

Emergency Procedures:

- On-call rotation
- Incident response plan
- Rollback procedures
- Communication plan

Database Maintenance:

- Index optimization
- Query optimization
- Data cleanup (soft-deleted records)
- Archive old data

Documentation Updates:

- Keep docs in sync with code
- Update API documentation
- Update deployment guides
- Update troubleshooting guides